

CYBERSECURITY INTERNSHIP — TASK 1

Basic Network Sniffer

Real-Time Packet Capture & Protocol Analysis using Python and Scapy

Intern Name	Mohamed unaiz
Internship	Code Alpha — Cybersecurity Internship
Task	Task 1 of 4 — Basic Network Sniffer
Tools Used	Python 3, Scapy, Kali Linux
Report Date	2026-06-16
Repository	Network Sniffer

Submission Summary

This report documents the design, implementation, and live execution of a custom Python-based network sniffer built using the Scapy library. The tool captures live traffic, classifies packets by protocol, and displays source/destination addresses, ports, MAC addresses, and payload contents in hex and ASCII form.

1. Objective

The goal of Task 1 was to build a working Python program capable of capturing live network traffic and analyzing the structure and content of each packet. This task introduces core networking concepts — how data flows across a network, how different protocols are structured, and how packet-level information can be used to understand or troubleshoot network behavior.

Specific Goals (as per CodeAlpha brief)

- Build a Python program to capture network traffic packets.
- Analyze captured packets to understand their structure and content.
- Learn how data flows through the network and the basics of common protocols.
- Use Scapy for packet capturing.
- Display source/destination IPs, protocols, and payloads.

2. Tools & Technologies Used

Component	Technology	Purpose
Language	Python 3.12	Core implementation language
Packet Library	Scapy 2.5+	Raw packet capture and dissection
OS Environment	Kali Linux (VM)	Provides raw socket access for sniffing
Privilege Model	sudo / root	Required for AF_PACKET raw socket access
Output	ANSI Terminal Colors	Color-coded, human-readable live display

3. Methodology & Implementation

3.1 Program Architecture

The sniffer is built around Scapy's `sniff()` function, which registers a callback (`process_packet`) executed for every captured packet. Each packet is inspected layer-by-layer — Ethernet → IP/IPv6/ARP → TCP/UDP/ICMP → DNS/HTTP → Raw payload — and the relevant fields are extracted and printed in a structured, color-coded format.

3.2 Protocol Detection Logic

A `detect_protocol()` function inspects the packet's layers in priority order — application-layer protocols (DNS, HTTP) are checked first, followed by transport-layer (TCP, UDP, ICMP), then ARP and IPv6. This ensures the most meaningful protocol label is shown for each packet.

```
# Protocol detection — checks highest-level protocol first
def detect_protocol(packet):
    if packet.haslayer(DNS): return "DNS"
    if packet.haslayer(HTTPRequest) or packet.haslayer(HTTPResponse):
        return "HTTP"
    if packet.haslayer(TCP): return "TCP"
    if packet.haslayer(UDP): return "UDP"
    if packet.haslayer(ICMP): return "ICMP"
    if packet.haslayer(ARP): return "ARP"
    return "OTHER"
```

3.3 Field Extraction

For each packet, the following fields are extracted where applicable:

- Layer 2: Source and Destination MAC addresses (Ethernet header)
- Layer 3: Source and Destination IP (IPv4/IPv6), TTL/hop limit
- Layer 4: Source/Destination ports, TCP flags decoded into names (SYN, ACK, FIN)
- Payload: Raw bytes shown as both hexadecimal and printable ASCII
- Application layer: DNS query/response names; HTTP method, host, and path

3.4 Command-Line Interface

The script accepts CLI arguments for flexibility:

```
-i, --interface  Network interface to sniff on (e.g. eth0, wlan0)
-c, --count      Number of packets to capture (0 = unlimited)
-f, --filter     BPF filter string (e.g. "tcp port 80", "udp port 53")
-v, --verbose    Enable verbose mode
```

```
# Example: capture 100 HTTP packets on eth0
sudo python3 task1_network_sniffer.py -i eth0 -c 100 -f "tcp port 80"
```

3.5 Session Summary

When the user presses Ctrl+C, a `print_summary()` handler is triggered via a SIGINT signal handler. It prints a per-protocol breakdown of all packets captured during the session, with an ASCII bar chart for visual clarity.

4. Live Execution & Results

The sniffer was executed on a Kali Linux virtual machine with root privileges, run with default settings (unlimited packets, default interface) and allowed to capture ambient LAN traffic for approximately 35 seconds before being stopped with Ctrl+C.

```
sudo python3 task1_network_sniffer.py
```

```
CodeAlpha - Basic Network Sniffer
Task 1 | Cybersecurity Internship

[*] Starting capture on default interface
[*] Count      : Unlimited
[*] Press Ctrl+C to stop and view summary

[#0001] [23:02:22.126] DNS
SRC → fe80::24c7:92ff:feb0:6f0e:5353   DST → ff02::fb:5353
MAC SRC: 26:c7:92:b0:6f:0e   MAC DST: 33:33:00:00:00:fb
INFO : Query: _spotify-connect._tcp.local
Packet size: 107 bytes

[#0002] [23:02:22.329] UDP
SRC → 10.235.124.172:37500   DST → 239.255.255.250:1900
MAC SRC: 26:c7:92:b0:6f:0e   MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 4D 2D 53 45 41 52 43 48 20 2A 20 48 54 54 50 2F 31 2E 31 0D 0A 48 4F 53 54 3A 20 32 33 39 2E 32 35
35 2E 32 35 35 2E 32 35 30 3A 31 39 30 30 0D 0A 4D 41 4E 3A 20 22 73 73 64 70 3A 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 167 bytes

[#0003] [23:02:26.831] UDP
SRC → 10.235.124.172:57621   DST → 10.235.124.255:57621
MAC SRC: 26:c7:92:b0:6f:0e   MAC DST: ff:ff:ff:ff:ff:ff
PAYLOAD: HEX : 53 70 6F 74 55 64 70 30 D9 FF D3 B6 F0 D9 CE EE 00 01 00 00 F3 0A C4 DE 1C 67 10 0F 5A 4A 86 E4 71
27 A8 90 AD 03 DE 81
ASCII: SpotUdp0.....g..ZJ..q'.....
Packet size: 82 bytes

[#0004] [23:02:53.024] UDP
SRC → 10.235.124.114:42843   DST → 95.216.192.15:123
MAC SRC: 08:00:27:d9:fb:6f   MAC DST: 2e:f7:79:ba:50:43
Packet size: 90 bytes

[#0005] [23:02:53.510] UDP
SRC → 95.216.192.15:123   DST → 10.235.124.114:42843
MAC SRC: 2e:f7:79:ba:50:43   MAC DST: 08:00:27:d9:fb:6f
Packet size: 90 bytes

[#0006] [23:02:58.096] ARP
SRC → 10.235.124.114:-   DST → 10.235.124.2:-
MAC SRC: 08:00:27:d9:fb:6f   MAC DST: 2e:f7:79:ba:50:43
Packet size: 42 bytes
```

Figure: Sniffer started — capturing packets #1 through #6. The tool immediately detects an mDNS/DNS query for '*_spotify-connect._tcp.local*', SSDP discovery (M-SEARCH) broadcasts over UDP, and ARP requests — all color-coded by protocol. The HEX and ASCII payload preview is visible for UDP packets.

```
[#0007] [23:02:58.100] ARP
SRC → 10.235.124.21- DST → 10.235.124.114+-
MAC SRC: 2e:f7:79:ba:50:43 MAC DST: 08:00:27:d9:fb:6f
Packet size: 60 bytes

[#0008] [23:03:11.873] ARP
SRC → 10.235.124.21- DST → 10.235.124.114+-
MAC SRC: 2e:f7:79:ba:50:43 MAC DST: ff:ff:ff:ff:ff:ff
Packet size: 60 bytes

[#0009] [23:03:11.873] ARP
SRC → 10.235.124.114- DST → 10.235.124.21-
MAC SRC: 08:00:27:d9:fb:6f MAC DST: 2e:f7:79:ba:50:43
Packet size: 42 bytes

^X@s
[#0010] [23:03:26.819] UDP
SRC → 10.235.124.172:57621 DST → 10.235.124.255:1900
MAC SRC: 26:c7:92:d8:6f:0e MAC DST: ff:ff:ff:ff:ff:ff
PAYLOAD: HEX : 53 70 6f 74 55 64 70 30 d9 ff d3 b6 f0 d9 ce ee 00 01 00 00 f3 0a c4 de 1c 67 10 0f 5a 4a 86 e4 71 27 a8 90 ad 03 de 81
ASCII: SpotUdp0.....g..ZJ..q'.....
Packet size: 82 bytes

[#0011] [23:03:26.920] ARP
SRC → 10.235.124.21- DST → 10.235.124.235+-
MAC SRC: 2e:f7:79:ba:50:43 MAC DST: ff:ff:ff:ff:ff:ff
Packet size: 60 bytes

^s
[#0012] [23:03:33.112] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes

[#0013] [23:03:34.113] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes

^X@s
[#0014] [23:03:35.113] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes
```

Figure: Continued capture — packets #7 through #14. Multiple ARP broadcast/reply pairs are visible between hosts, followed by repeated SSDP discovery UDP packets to the multicast address 239.255.255.250:1900, with the full 'M-SEARCH * HTTP/1.1' SSDP payload visible in ASCII.

```
MAC SRC: 08:00:27:d9:fb:6f MAC DST: 2e:f7:79:ba:50:43
Packet size: 42 bytes

^X@s
[#0010] [23:03:26.819] UDP
SRC → 10.235.124.172:57621 DST → 10.235.124.255:1900
MAC SRC: 26:c7:92:d8:6f:0e MAC DST: ff:ff:ff:ff:ff:ff
PAYLOAD: HEX : 53 70 6f 74 55 64 70 30 d9 ff d3 b6 f0 d9 ce ee 00 01 00 00 f3 0a c4 de 1c 67 10 0f 5a 4a 86 e4 71 27 a8 90 ad 03 de 81
ASCII: SpotUdp0.....g..ZJ..q'.....
Packet size: 82 bytes

[#0011] [23:03:26.920] ARP
SRC → 10.235.124.21- DST → 10.235.124.235+-
MAC SRC: 2e:f7:79:ba:50:43 MAC DST: ff:ff:ff:ff:ff:ff
Packet size: 60 bytes

^s
[#0012] [23:03:33.112] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes

[#0013] [23:03:34.113] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes

^X@s
[#0014] [23:03:35.113] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes

[#0015] [23:03:36.114] UDP
SRC → 10.235.124.235:60461 DST → 239.255.255.250:1900
MAC SRC: 2c:3b:78:0d:46:f1 MAC DST: 01:00:5e:7f:ff:fa
PAYLOAD: HEX : 40 20 53 45 41 52 43 48 20 2a 20 48 54 5a 50 2f 31 2e 31 0d 0a 48 4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 35 30 3a 31 39 30 30 00 0a 4d 41 4e 3a 20 22 73 73 64 70 3a 64 69 73 63
ASCII: M-SEARCH * HTTP/1.1..HOST: 239.255.255.250:1900..MAN: "ssdp:disc
Packet size: 212 bytes

^C

Session Summary | Total Packets: 15

UDP      9
ARP      5
DNS      1
```

Figure: End of capture — packet #15, followed by the Ctrl+C-triggered session summary showing 15 total packets: 9 UDP, 5 ARP, 1 DNS, displayed with a color-coded ASCII bar chart.

5. Results & Analysis

5.1 Session Statistics

Protocol	Packets	Observation
UDP	9	Dominated by SSDP (UPnP discovery) and Spotify Connect mDNS-like UDP broadcasts
ARP	5	Address resolution between local hosts (10.235.124.2 and 10.235.124.114)
DNS	1	mDNS query for '_spotify-connect._tcp.local' — local service discovery
TOTAL	15	Captured over ~35 seconds of ambient LAN traffic

5.2 Key Observations

- Even an "idle" network generates significant background traffic — ARP resolution, SSDP/UPnP discovery broadcasts, and mDNS service announcements account for 100% of the captured packets in this short session.
- SSDP M-SEARCH packets sent to multicast 239.255.255.250:1900 demonstrate how devices on a LAN continuously broadcast discovery requests (used by smart TVs, media servers, IoT devices).
- The hex+ASCII payload view makes it possible to read plaintext protocol data directly — the full "M-SEARCH * HTTP/1.1 ... HOST: 239.255.255.250:1900" string is readable in the ASCII column.
- The mDNS query for '_spotify-connect._tcp.local' reveals a Spotify-enabled device broadcasting its service availability — a useful example of how passive sniffing can reveal device information.

5.3 Educational Value

This exercise demonstrated foundational networking concepts in practice: the layered structure of protocols (Ethernet → IP/ARP → UDP/TCP → Application data), the prevalence of broadcast/multicast discovery on local networks, and the level of detail visible to anyone with raw packet capture access — reinforcing why encrypted protocols (HTTPS, DoH/DoT) and network segmentation are important security controls.

6. Challenges Faced & Learnings

- Root privilege requirement: Scapy's raw socket capture requires root access on Linux. Resolved by running with sudo, with an alternative documented using Linux capabilities (setcap cap_net_raw+ep) for non-root use.
 - Protocol classification ordering: Some packets match multiple layers (e.g., a DNS packet is also UDP/IP). The detection function checks application-layer protocols first to ensure the most informative label is shown.
 - Payload readability: Raw bytes are not human-readable by default. Implementing a combined hex + ASCII view (replacing non-printable bytes with '.') made the payload section immediately useful.
 - Handling Ctrl+C gracefully: A SIGINT signal handler was registered so stopping the capture produces a clean summary rather than an unhandled exception.
-

7. Conclusion

Task 1 was successfully completed. The Python/Scapy-based network sniffer captures and classifies live traffic across multiple protocols (ARP, UDP, DNS, TCP, ICMP, HTTP), displays source/destination addressing at both link and network layers, and provides a readable hex/ASCII payload view. The live execution session captured and correctly classified 15 real packets of LAN traffic, producing an accurate session summary. This task provided hands-on experience with packet-level network analysis — a foundational skill for network security monitoring, intrusion detection, and traffic forensics.
